Compound Protocol Proof

installer

March 11, 2026

Chapter 1

Compound Protocol Consistency

We prove Theorem 6.1 described in Paper Section 6.5. Our theorem states that Compound Protocols (consisting of an SWMR Global Protocol connecting two (SWMR or RCC-Style) Cluster Protocols via translation shims) enforce the CMCM (Compound Memory Consistency Model) PPOs (Preserved Program Orderings). In this document, our Lean mechanization of a Compound Protocol is introduced in Section 1.7. When referring to our paper's sections, we will use "Paper Section". For sections of this document, we will use "Section".

We'll (re-)motivate the definitions and components of the proof in a top-down manner, with a running example. We'll note which definitions are from the paper, and indicate where those definitions are in the Lean mechanization. First, we (re-)present the idea of what Theorem 6.1 states, and how it's proven.

Paper Section 6.4 provides intuition on Theorem 6.1's proof. It uses the example of an Acquire ordered before a weakly ordered read in an RCC-style protocol as an ordered PPO pair. An Acquire followed by a weakly ordered read is a PPO pair because the Acquire indicates subsequent reads are forced to read from the upper level memory after the Acquire reads from the upper level memory. This means that the acquire linearizes in memory before the subsequent read linearizes in memory. Section 1.1, Definition 6, introduces our PPO mechanization in Lean. We'll briefly re-introduce the idea of a cache request "linearizing" from Paper Section 5.3's Paper Definition 5.4 and 5.5. Section 1.1, Definitions 4 and 5, introduces our linearization events of a cache request event in Lean. Section 1.6 introduces temporal relations between events (ordered before, and encapsulates).

The proof proceeds by considering the cases of when does a cache request event e_{req} linearizes. e_{req} may linearize prior to, during, or after its request in cache. When e_{req} linearizes depends on two factors:

1. The kind of request it is, based on its Consistency Label.
2. If e_{req} is made on cache *State* with coherent Permissions.

Requests (Paper Definition 5.2) are introduced in Section 1.1, Definition 2. State and having Permissions are introduced in Section 1.1, Definitions 1 and 3 respectively. The notion of the state a cache is made on is introduced in Section 1.1, Definition 7. The state of a cache before a cache request event depends on the events before it, at its cache's address, of an execution. Note that we refer to an execution as a "Behaviour".

A cache request event e_{req} may linearize by accessing its cluster's directory. If the directory has permissions at the Interconnect interface (Global Protocol), then e_{req} linearizes at its directory access. Otherwise, the Translation Shim between Cluster and Global protocols then

translate e_{req}'s directory request to a Global Protocol access. Then e_{req} linearizes when it reaches the Global Protocol's Directory. Section 1.5 introduces the Lean mechanization of the Translation Shim.

We break down the cases of proving the PPO requests linearize in order into linearization-ordering lemmas (introduced in Sections 1.12, and 1.13).

The linearization-ordering lemmas are supported by more lemmas about the cases of a cache request e_{req}'s linearization event, and axioms across Sections 1.2, 1.3, 1.4, 1.5, 1.9, 1.10, and 1.11.

There are more supporting lemmas that reason about events, execution traces ("Behaviours") across files BehaviourHelpers.lean, BehaviourRelationProofs.lean, ProofBasicHelperLemmas.lean, and a few in definition files such as EventRelations.lean.

1.1 Definitions

Definition 1 (State (Paper Def 5.1))**.** (Line 90 in States.lean.) A *state* consists of a tuple of a permissions level p (none, read, or write) and a coherence flag c (coherent or non-coherent). The five states are:

- SW (Shared-Write): write permissions, coherent,
- MR (Multiple-Read): read permissions, coherent,
- Vd (Valid-dirty): write permissions, non-coherent,
- Vc (Valid-clean): read permissions, non-coherent,
- I (Invalid): no permissions, non-coherent.

Definition 2 (Abstract Request (Paper Def 5.2))**.** (Line 19 in Requests.lean.) An *abstract request* consists of:

- A read/write operation,
- A coherence flag,
- A consistency label (SC, Release, Acquire, or Weak).

See ValidRequest at line 66, defining a Request and restricting disallowed Requests such as a Coherent Acquire or Write Acquire.

Definition 3 (Request's Required State (Paper Def 5.3))**.** (Line 503 in BehaviourRelationDefs.lean.) A request event e *has permissions* if the cache state it is made on is at least its Minimum Required State (MRS). Let's start by defining a simple Required State (RS) of a request 'r'. Let the read/write operation 'rw' and coherence flag 'c' of 'r' be the State tuple ('rw', 'c') to define its RS.

For coherent requests (ex. a SC Write from a SWMR protocol, or Coherent Release Writes from RCC-O) we use the simple RS definition to define the MRS. This is reflected in the first case in the inductive 'Behaviour.reqHasPerms'.

For non-coherent acquire/release/weak writes from RCC protocols, the request's MRS additionally requires the state is coherent. This is reflected in the second case in the inductive 'Behaviour.reqHasPerms'.

For non-coherent weak read/write requests uses the simple RS definition and specifies the cache state is Read-Non-Coherent. The state is not Vd. This is reflected in the third case in the inductive 'Behaviour.reqHasPerms'.

Definition 4 (Local Linearization Point (Paper Def 5.4))**.** (Line 1083 in BehaviourRelationProofs.lean.) The *linearization event* of a request event e_{req}. Either the cluster cache request event linearizes in the cache (on a state with coherent permissions), or at the directory (without coherent permissions). Linearizing at the directory means either linearizing before, during, or after the request (as per inductive 'Behaviour.dirAccessOfRequest' on line 569 of BehaviourRelationDefs.lean). More succintly in 'Behaviour.linearizationEventOfRequest':

- If e_{req} is made on a coherent state with sufficient permissions, the linearization event is e_{req} itself (`requestLin`).
- Otherwise, the linearization event is the directory event e_{dir} corresponding to e_{req} (`dirLin`).

Definition 5 (Global Linearization Point (Paper Def 5.5))**.** (Line 139 in CompoundLinearization.lean.) A cluster request's cluster linearization event classifies where a cluster cache request globally linearizes:

- **At the Cluster Cache** (`clusterCacheLin`): the request has permissions and linearizes locally; the linearization event is the cache event itself.
- **At the Cluster Directory or Beyond** (`clusterDirLin`): the request does not have permissions locally and must go through the directory (and possibly the global protocol). The request then either globally linearizes at the directory (with global permissions already) or linearizes at the global directory (needs to access the global directory for permissions).

In the case where the cluster request linearizes at the cluster directory or global protocol is defined by the inductive 'CompoundProtocol.clusterDirectoryLinearizationEvent' at line 98 in the same file.

Definition 6 (Request PPOs)**.** (Line 3 in RequestPPOs.lean.) A simple proposition between two Requests, stating if the two requests are a PPO pair.

Definition 7 (Cache State a Cache Request Event is Made on)**.** (Line 15 in BehaviourRelationDefs.lean.) This definition refers to the state a cache request event arrives at a cache entry on. This def uses other definitions and the cache ordered axiom (Well-Formedness Axiom 1) to project the events at a cache in a list following the order they're made. The state before an event e_{req} is the state after the list of events up to e_{req}.

There is a similar definition for directory events (events at a directory) on line 17 of BehaviourRelationDefs.lean.

1.2 Well-Formedness Axioms

Definition 8 (Ordered Directory Events (Well-Formedness Axiom 1))**.** (Line 305 in EventRelations.lean.) Events at the same directory address are totally ordered: for any two directory events de_1, de_2 at the same address, either de_1 is ordered before de_2 or vice versa. This is stated by the structure fields in 'DirectoryEvent.AreOrdered', 'sameDirectoryEntry' and 'ordered'.

Definition 9 (Ordered/Encapsulated Cache Events (Well-Formedness Axiom 2))**.** (Line 341 in EventRelations.lean.) For any two cache events e_1, e_2 at the same cache entry, either e_1 is encapsulated by e_2, e_2 is encapsulated by e_1, e_1 is ordered before e_2, or e_2 is ordered before e_1. This is stated by the structure fields in 'CacheEvent.AreOrdered', 'sameCacheEntry' and 'ordered'.

Definition 10 (Cache Events Encapsulate Corresponding Directory Event (Well-Formedness Axiom 3))**.** (Line 55 in BehaviourRelationDefs.lean.) A cache operation event encapsulates a corresponding directory event, whose request field, address, downgrade flag, and protocol match the cache event. This is reflected in the fields of structure 'Behaviour.requestDirectoryEvent'.

Definition 11 (Cache Events Access Dir If Insufficient Permissions (Well-Formedness Axiom 4))**.** (Line 209 in BehaviourRelationDefs.lean.) If a cache event does not have sufficient permissions (its state is below the Minimum Required State), it accesses the directory. The inductive 'Behaviour.requestAccessesDirectory' describes the cases of different request events encapsulating corresponding directory accesses.

Definition 12 (Cache Events of Directory Responses Arrive In-Order (Well-Formedness Axiom 5))**.** This axiom states that cache events of directory responses arrive in-order. It does not have a separate Lean declaration; it is realized by Axiom 1 (8) combined with the Downgrade Abstract State Invariants — downgrades are encapsulated by a directory event, so they are in order because directory events are ordered.

Definition 13 (Directory Responses to Caches Are Ordered (Well-Formedness Axiom 6))**.** (Line 532 in EventRelations.lean.) A request event encapsulates a grant event that is ordered after the directory event, and the grant ends the request. This is reflected in the fields of structure 'Event.encapGrantAfterDirEvent'.

Definition 14 (Cache with Perms Has Predecessor That Obtained Perms (Well-Formedness Axiom 7))**.** (Line 494 in BehaviourRelationProofs.lean.) If a request event e_{req} has permissions, there exists an immediate predecessor that obtained those permissions by accessing the directory, and all intermediate events maintain sufficient state for e_{req}. This is reflected by the fields in structure 'Behaviour.exists_predecessor_setting_state'.

Definition 15 (Weak Request on Vd Has Successor WB or Get SW (Well-Formedness Axiom 8))**.** (Line 508 in BehaviourRelationProofs.lean.) A weak request on Write-Non-Coherent state ordered before a write-back evict linearizes at the write-back evict. There exists an immediate bottom successor that encapsulates a corresponding directory event. This is reflected by the fields in structure 'Behaviour.exists_vd_successor_wb_or_get_sw'.

Definition 16 (Acquire Invalidation OB Weak Read Predecessor (Well-Formedness Axiom 9))**.** (Line 8 in Protocol.lean.) A non-coherent weak read ordered after a cache entry invalidation (from an acquire) will linearize after the invalidation: the invalidation event is ordered before the predecessor that gets permissions. This is reflected by the parameters in def 'CompoundProtocol.acquire_invalidation_OrderedBefore_weak_read_predecessor_that_gets_perms'

1.3 Abstract State Invariants

Definition 17 (NC Weak Write Linearizes After Cache Event (Abstract State Invariant 1))**.** (Line 233 in BehaviourRelationDefs.lean.) Non-coherent weak write request linearizes after its cache event. Its cache entry in Vd state means there's an immediate successor that is either a VdWriteBack or a CoherentWrite (which would linearize the weak write). This is reflected in the fields of structure 'Behaviour.vdCacheEntryWriteBackLater'. Field 'vdStateAfterEvent' states the non-coherent weak write results in cache state Vd (Non-Coherent Write). Field 'wbImmPred' states the weak write an immediate successor as described above.

Definition 18 (Coherent Write Dir Events Result in Downgrades (Abstract State Invariant 2))**.** (Line 294 in BehaviourRelationDefs.lean.) Coherent write directory events result in downgrades. A coherent write request to the directory results in downgrades at other caches. On SW state, it forwards to the owner; on MR state, it forwards to all sharers. This is reflected in inductive 'Behaviour.coherentWriteAtDirectoryEncapDowngrades'. Case 'cWriteOnSW' represents forwarding a coherent write to the owner cache. Case 'cWriteOnMR' represents forwarding a coherent write to the sharer caches.

Definition 19 (Coherent Read Dir on SW Downgrades Owner (Abstract State Invariant 3))**.** (Line 324 in BehaviourRelationDefs.lean.) Coherent read directory event on Write-Coherent state downgrades the owner. This is reflected in structure 'Behaviour.coherentReadDowngradeOthers' at line 316 used by the definition of 'Behaviour.coherentReadDirDowngradeOthers' at line 324. Field 'downgradeOtherCaches' specifically downgrades the owner on Write-Coherent state, with a coherent-read downgrade.

Definition 20 (NC Requests on SW Downgrade Owner (Abstract State Invariant 4))**.** (Line 345 in BehaviourRelationDefs.lean.) Non-coherent write/read requests on Write-Coherent state will downgrade the owner. This is reflected in the structure 'Behaviour.nonCoherentReqOnSWDowngradeOthers' used by definition 'Behaviour.nonCoherentRequestDowngradeOthers'. Field 'fwdPrevOwner' specifically downgrades the owner on Write-Coherent state.

Definition 21 (Protocol Enforces SWMR (Abstract State Invariant 5))**.** (Line 96 in SWMR.lean.) The protocol enforces Single-Writer Multiple-Reader (SWMR): at any point in time, cache states across the protocol instance cannot simultaneously have both SW and MR, and at most one cache can be in SW state. This is reflected in definition 'SWMR', where there cannot be both a Write-Coherent (SW) and Read-Coherent (MR) cache simultaneously. The number of Write-Coherent caches must be at most 1.

1.4 Consistency Label Axioms

Definition 22 (Acquire Invalidates Other Entries (Consistency Label Axiom 1))**.** (Line 374 in BehaviourRelationDefs.lean.) Acquire cache operations access the directory, then invalidate Read-Non-Coherent cache entries. The acquire cache operation encapsulates Vc (Read-Non-Coherent) evicts to cache entries at other addresses after accessing the directory. The structure 'Behaviour.nonCoherentReqOnSWDowngradeOthers' field broadcastInval invalidates the other cache entries after the acquire's directory access event.

Definition 23 (NC Release Writes Back Other Entries (Consistency Label Axiom 2))**.** (Line 385 in BehaviourRelationDefs.lean.) Non-coherent release cache operations write back Write-Non-Coherent entries, then access the directory. This is achieved by using Vd (Write-Non-Coherent) write-back events encapsulated by the release, ordered before the directory event. The structure 'Behaviour.relWriteBackOtherEntries' field broadcastWB writes back the other cache entries before the release's directory access.

Definition 24 (L-RCC Coherent Release Downgrade Write-Back (Consistency Label Axiom 3))**.** (Line 407 in BehaviourRelationDefs.lean.) L-RCC variant with Coherent Release Write and Non-Coherent Weak Write. When receiving a downgrade to a cache line with Coherent-Write state, the downgrade writes back Vd entries at other addresses. This is the L-RCC variant where the protocol interface contains both a coherent release and a non-coherent weak write. This is reflected by the broadcastWBs field in the structure 'Behaviour.coherentRelDowngradeWriteBackOthers'. The fields 'cRelInPI' and 'ncWeakWriteInPI' state the protocol's request interface contains a Coherent Release and Non-Coherent Weak Write respectively, as per L-RCC.

1.5 Shim Axioms

Definition 25 (Cluster to Global Propagation Shim (Shim Axiom 1))**.** (Line 128 in BehaviourShim.lean.) Cluster directory events are translated to request events at the corresponding cache in the global protocol. Two cases:

- **encapGlobalCache**: the global cache lacks permissions, so the directory event encapsulates a global cache access.
- **noGlobalCache**: the global cache already has permissions (so no global directory access), and a prior global cache event with permissions exists.

These are the two cases of the inductive 'Behaviour.Shim.ClusterToGlobal'.

Definition 26 (Global to Cluster Propagation Shim (Shim Axiom 2))**.** (Line 903 in BehaviourShim.lean.) Downgrades at a global cache are translated to cluster directory accesses. The translation depends on whether the cluster protocol has both coherent write and read (an SWMR protocol), or only non-coherent reads (a RCC-style protocol). This is reflected in the inductive 'Behaviour.Shim.GlobalToCluster'.

1.6 Ordered Before and Encapsulates Relations

Definition 27 (Ordered Before)**.** An event e_1 is *ordered before* e_2 if e_1 completes before e_2 starts: e_1.oEnd $< e_2$.oStart. This is defined on line 12 in EventRelations.lean as 'Event.OrderedBefore'.

Definition 28 (Encapsulates)**.** An event e_1 *encapsulates* e_2 if e_1 starts before e_2 starts and e_2 ends before e_1 ends: e_1.oStart $< e_2$.oStart $\wedge$ e_2.oEnd $< e_1$.oEnd. This is defined on line 6 in EventRelations.lean as 'Event.Encapsulates.

1.7 Compound Protocol

Definition 29 (Compound Protocol)**.** A *compound protocol* bundles:

- A global protocol and two cluster protocols,
- Shim axioms for translating between cluster and global protocols,
- A linearization function assigning each request event a linearization event,
- A compound linearization event function,
- Axioms: the global protocol is SWMR, acquire events on a shared-write cannot linearize at cache, and weak write / non-coherent release pairs cannot both linearize at cache.

1.8 Compound (Global) Linearization Order

Definition 30 (Lazy Compound Linearization Order)**.** A *lazy compound linearization order* between a linearization event e_{lin_1} and a second event e_2 holds if: for every immediate bottom predecessor e_3 of e_2's directory event at the same address, e_{lin_1} finishes before e_3's directory event.

Definition 31 (Compound Linearization Order)**.** For a PPO pair (e_1, e_2), *compound linearization order* holds if either their compound linearization events are ordered (e_{lin_1} is ordered before e_{lin_2}), or the lazy compound linearization order holds between e_{lin_1} and e_2.

1.9 Auxiliary Definitions Organized in Structures

Definition 32 (Coherent Write Succeeded by Directory Request)**.** Structure capturing the auxiliary data for a coherent write event that is succeeded by a directory request.

Definition 33 (RCCO-Style Downgrade)**.** Structure capturing the chain of events in an RCCO-style downgrade: a directory event, a downgrade event, and a write-back event.

Definition 34 (Write-Back at Event)**.** Structure recording a VdWriteBack at a given event's entry.

1.10 Contradiction Lemmas

Lemma 35 (Contradiction: Has Perms and No Perms)**.** *A coherent event cannot simultaneously have and lack permissions.*

Lemma 36 (Contradiction: NC Release Has Perms and No Perms)**.** *A non-coherent release request cannot simultaneously have and lack permissions.*

Lemma 37 (Contradiction: Weak Write Has Perms and No Perms)**.** *A weak write request cannot simultaneously have and lack permissions.*

Lemma 38 (Contradiction: Acquire Has Perms and No Perms)**.** *An acquire request cannot simultaneously have and lack permissions.*

Lemma 39 (Contradiction: Coherent Has Perms and No Perms)**.** *A coherent request cannot simultaneously have and lack permissions.*

1.11 Encapsulation Lemmas

Lemma 40 (Directory Lin Encapsulates Global Directory Lin)**.** *If a cluster directory event encapsulates the global directory linearization event, then the encapsulation relation is preserved through the linearization chain.*

Lemma 41 (Event Encaps Dir Lin Encaps Global Dir Lin)**.** *If a cache event encapsulates a cluster directory event, and the cluster directory event encapsulates the global directory linearization event, then the cache event encapsulates the global directory linearization event.*

Lemma 42 (Request Encapsulates Compound Linearization Event)**.** *A request event encapsulates its compound linearization event (generic version for all request types).*

Lemma 43 (NC Release Encapsulates Compound Lin Event)**.** *A non-coherent release request encapsulates its compound linearization event.*

Lemma 44 (Acquire Encapsulates Compound Lin Event)**.** *An acquire request encapsulates its compound linearization event.*

Lemma 45 (Coherent Encapsulates Compound Lin Event)**.** *A coherent request encapsulates its compound linearization event.*

1.12 Linearization Ordering Lemmas

Lemma 46 (Ordered Before and Linearizes at Cache)**.** *If both events linearize at cache and e_1 is ordered before e_2, then compound linearization order holds. The linearization events are the events themselves.*

Lemma 47 (Cluster Dir Encap Global Lin at Dir)**.** *A cluster directory event encapsulates its compound linearization event when linearizing at directory.*

Lemma 48 (Weak Write/Read and NC Release Linearize at Directory)**.** *If a weak write or read and a non-coherent release both linearize at directory, their linearization events are ordered.*

Lemma 49 (Acquire and Weak Request Linearize at Directory)**.** *If an acquire and a weak request both linearize at directory, their linearization events are ordered.*

Lemma 50 (Weak Write Ordered Before VdWriteBack)**.** *A weak write that is ordered before a VdWriteBack linearizes before the write-back.*

Lemma 51 (State Before Vd of No Between Events)**.** *If there are no between-events on a Vd entry, the state before the Vd event produces a Vd state.*

Lemma 52 (Lazy Coherent Release Ordering (Helper))**.** *Helper lemma for lazy coherent release ordering.*

Lemma 53 (Lazy Coherent Release Ordering)**.** *Establishes the lazy linearization ordering for coherent releases.*

Lemma 54 (Weak Request to Dir and Coherent Release at Cache)**.** *When a weak request linearizes at directory and a coherent release linearizes at cache, compound linearization order holds.*

Lemma 55 (Contradiction: Weak Write Encapsulating Directory Event)**.** *A contradiction arises if a weak write encapsulates its corresponding directory event.*

1.13 Per-Case Compound Linearization Lemmas

Lemma 56 (CLO: Two Events Encapsulating Their Lin Events)**.** *Handles PPO cases where both events are non-coherent release, acquire, or coherent (e.g. SC×SC, Rel×Acq, Acq×Rel). Shows that each event encapsulates its compound linearization event, so ordered-before is preserved to the linearization events.*

Lemma 57 (CLO: Weak Read and NC Release)**.** *Weak read ordered before a non-coherent release satisfies compound linearization order.*

Lemma 58 (CLO: Weak Write or Read and NC Release)**.** *Generic lemma: weak write or read before a non-coherent release. Handles all sub-cases (cache×cache, dir×cache, cache×dir, dir×dir linearization).*

Lemma 59 (CLO: Weak Request and NC Release)**.** *Weak non-coherent request ordered before a non-coherent release satisfies compound linearization order.*

Lemma 60 (CLO: Acquire and Weak Request)**.** *Acquire ordered before a weak non-coherent request satisfies compound linearization order.*

Lemma 61 (CLO: Weak Request and Coherent Release)**.** *Weak non-coherent request ordered before a coherent release satisfies compound linearization order.*

1.14 Main Results

Lemma 62 (PPO Cluster Events Satisfy Compound Linearization)**.** *For same-cluster PPO cache events satisfying ordered-before, the compound linearization order holds. The proof is by exhaustive case match on the request types of e_1 and e_2, dispatching to the appropriate per-case lemma.*

Theorem 63 (Compound Protocol Enforces Consistency (Theorem 6.1))**.** *(Line 8 in CompositionalMCM.lean.) For any PPO pair of non-downgrade cache events e_1, e_2 at the same cache with different addresses in a behaviour b of a compound protocol: if e_1 is ordered before e_2, then compound linearization order holds between e_1 and e_2.*